

SOKOBAN

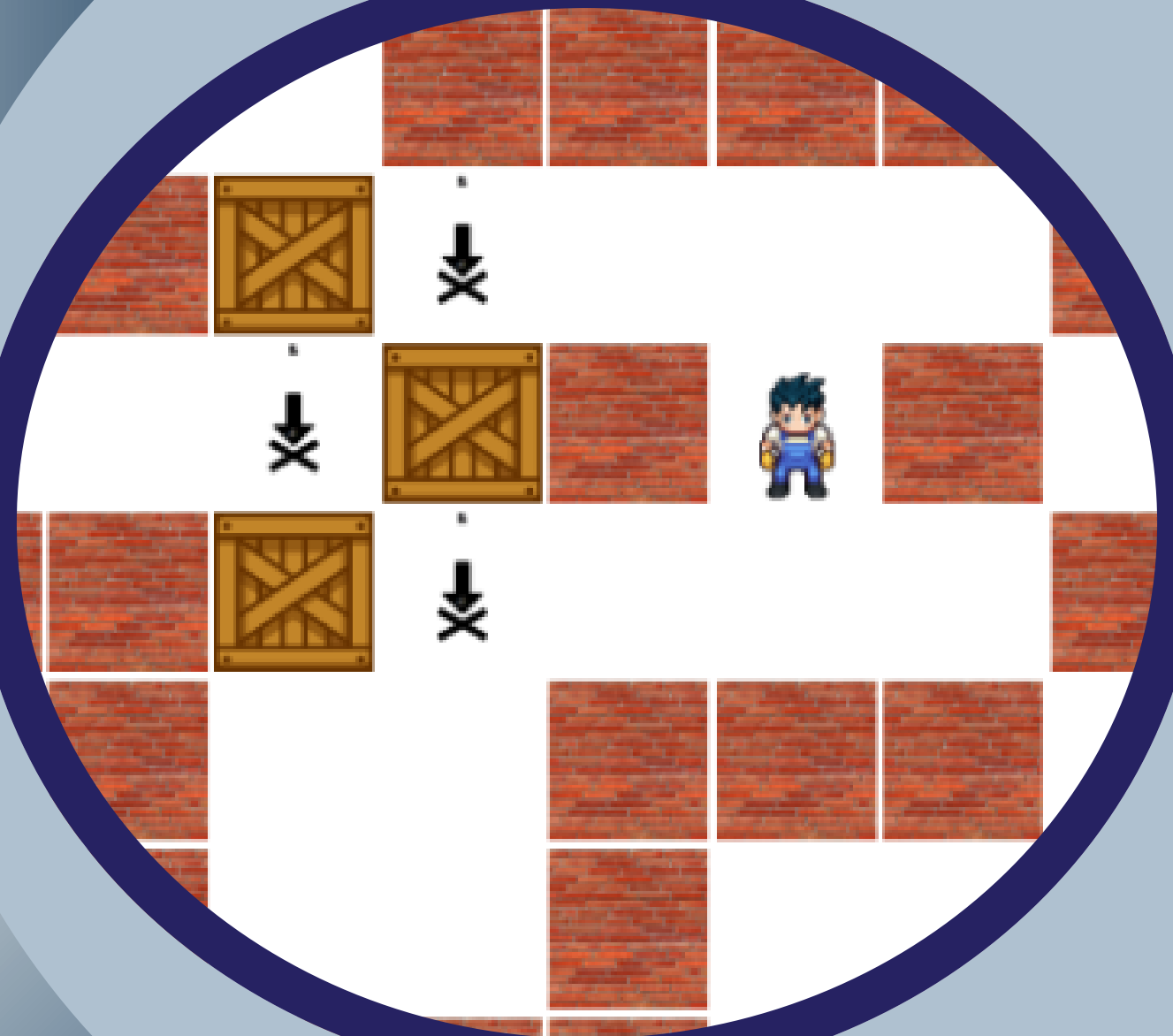
https://github.com/Fanomezantsoa-Beatrice/Myg_Angela_jingyi_Amine

Presented by

HUANG Jingyi

BENHAMMANE Amine

FANOMEZANTSOA Angela



WHAT IS A SOKOBAN ?

Sokoban :

- Pousser des caisses jusqu'aux cibles
- Pas de tirage, un seul joueur
- Puzzle basé sur les mouvements

Intérêt en Pharo :

- Bonne modélisation objet
- Simulation simple du jeu
- Facile à étendre (nouveaux challenges)



ARCHITECTURE GÉNÉRALE DE MYG

Grille

La grille représente le niveau sous forme de cases, chacune associée à un type de tuile.

Entités

Les éléments du jeu (Player, Box, Wall, Tiles...) interagissent entre eux dans cette grille.

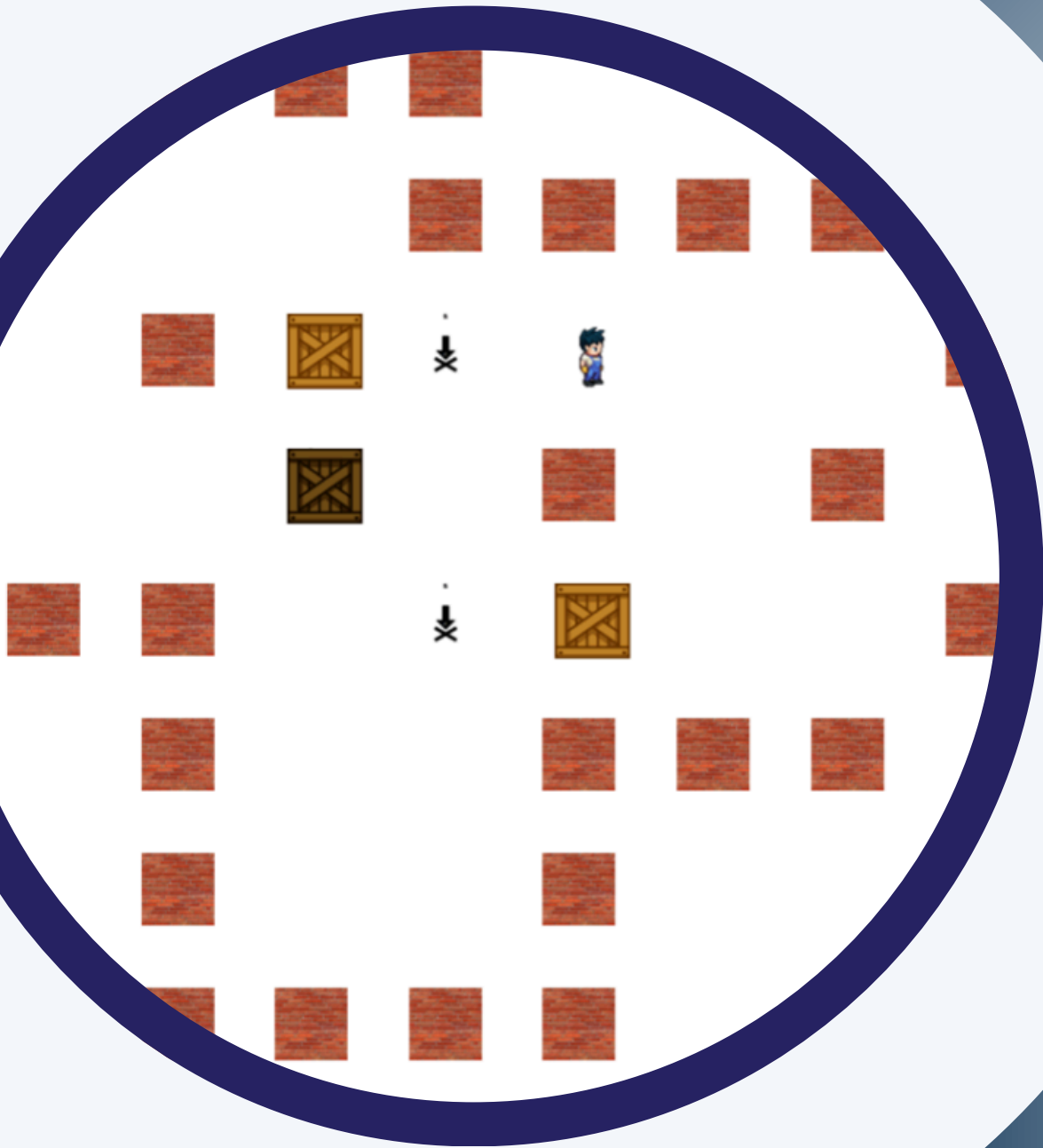
Moteur de jeu

Il gère les règles : déplacements, collisions et validité des actions du joueur.

Interface

L'UI affiche simplement l'état du plateau et met à jour la scène à chaque action.

Téléportation (Teleport Tiles)



Concept

Deux tuiles spéciales, appelées Teleport Tiles, permettent de transporter instantanément le joueur d'un point à un autre.

Fonctionnement

Quand le joueur entre sur la première tuile, il réapparaît immédiatement sur la seconde, et inversement.

Règle

Seul le joueur est téléporté, les caisses et obstacles restent à leur place.

Avantage

Cela crée de nouveaux défis et stratégies dans le jeu, en enrichissant le gameplay classique.

IMPLÉMENTATION : BRINGIN:

Intercepter l'arrivée

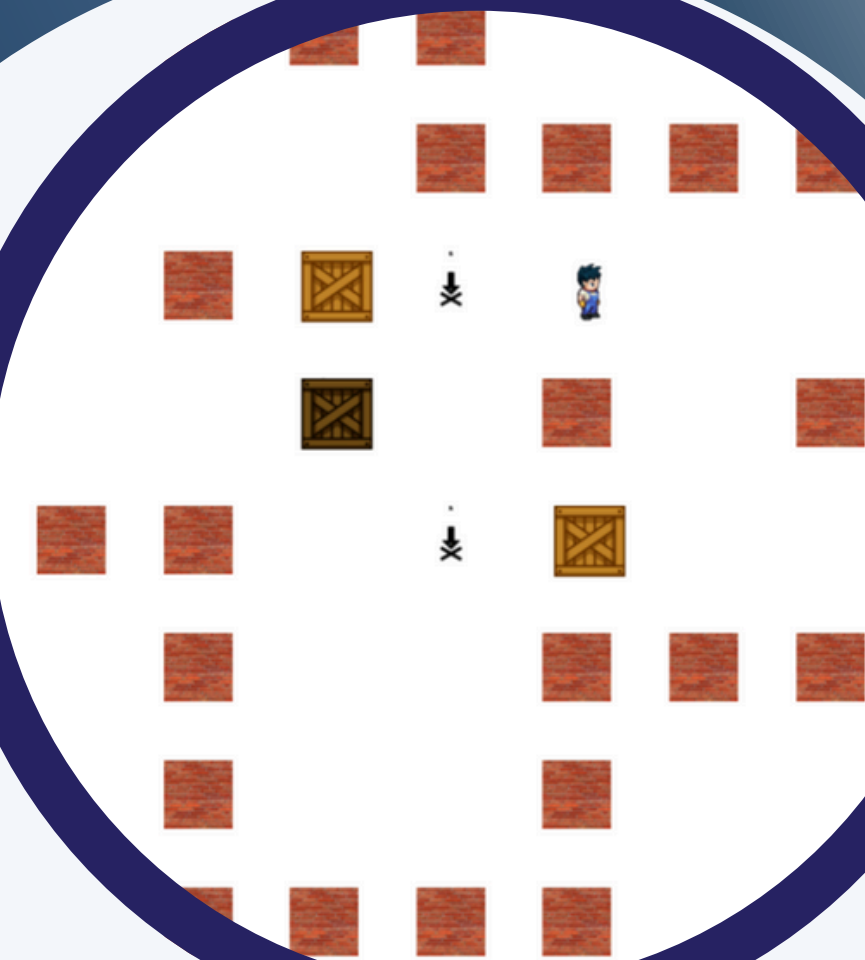
La méthode bringIn: est redéfinie pour gérer le comportement spécifique du téléport.

Filtrage

Seuls les joueurs sont téléportés ; les caisses passent par la logique normale.

Flux

- Vérifie que l'objet est un joueur.
- Appelle le super bringIn: pour gérer les mouvements et push.
- Si le mouvement réussi, appelle teleportPlayer: pour la téléportation.



IMPLÉMENTATION : TELEPORTPLAYER:

Déplacement atomique :

teleportPlayer: déplace le joueur vers l'autre téléporteur et met à jour la board.

Évitement de boucle infinie :

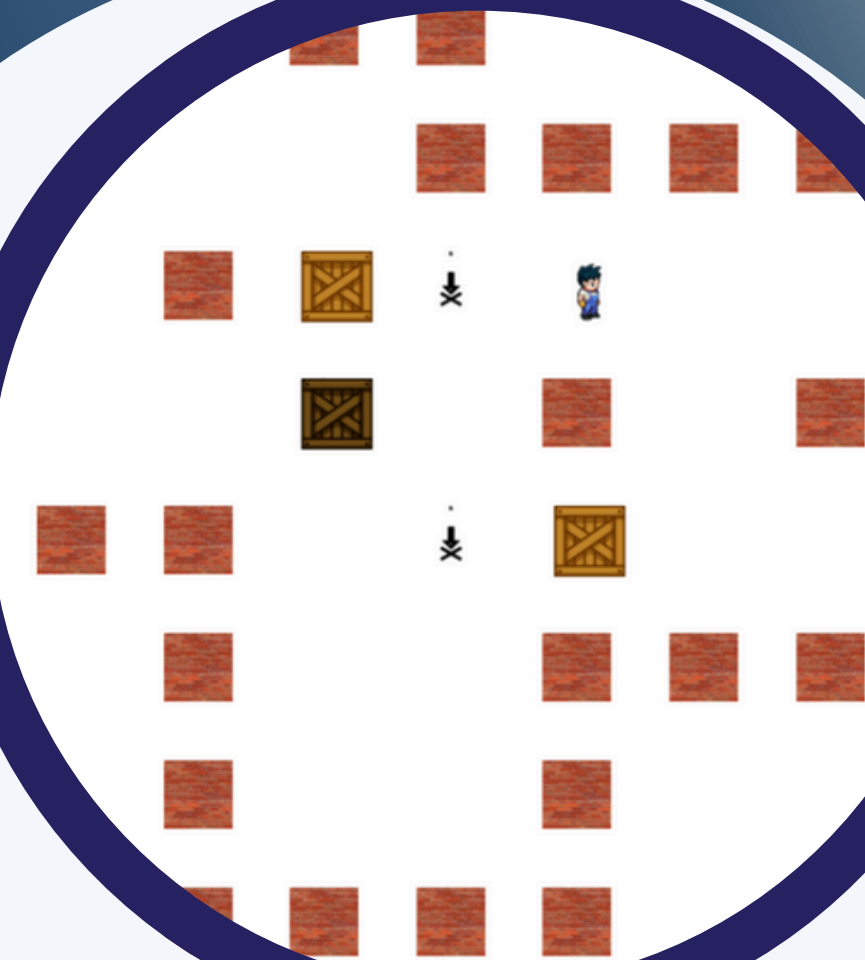
Un flag teleportCooldown empêche une téléportation répétée immédiate.

Recherche de l'autre téléporteur :

La méthode otherTeleport parcourt le plateau pour trouver la deuxième tuile.

Polymorphisme :

Le message isTeleport permet de gérer ce comportement spécifique sans casser le moteur de jeu existant.



POLYMORPHISME : ISTELEPORT

Concept

Le polymorphisme permet à chaque tuile de se comporter différemment selon son type.

Spécialisation :

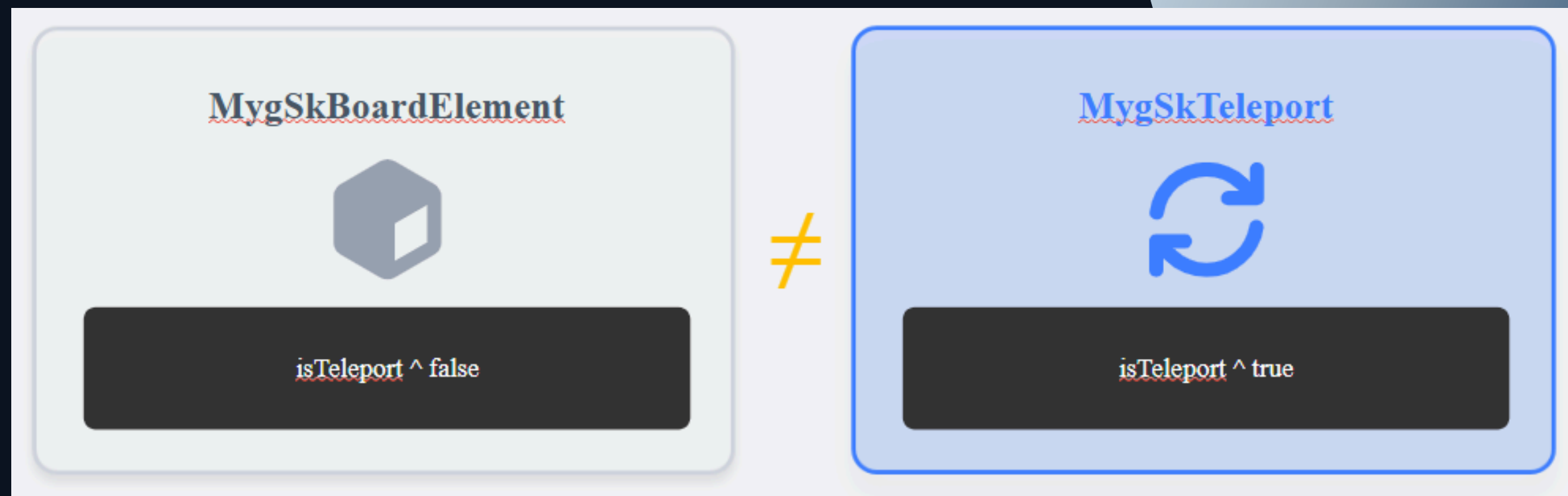
La tuile de téléport redéfinit la méthode isTeleport pour signaler qu'elle est spéciale.

Avantage

On n'a pas besoin de tester la classe du joueur ou de la tuile avec des if complexes.

Impact :

Le moteur de jeu reste générique et extensible : ajouter de nouvelles tuiles spéciales est simple et sûr.



OTHERTELEPORT : TROUVER L'AUTRE TÉLÉPORTEUR

Concept

Chaque téléporteur doit connaître son partenaire pour transporter le joueur. La méthode `otherTeleport` permet de retrouver l'autre téléporteur sur la grille.

Implémentation

La méthode parcourt toutes les tuiles du plateau et retourne celle qui est marquée comme téléporteur partenaire. Cela évite d'utiliser des coordonnées codées en dur ou des variables globales.

Robustesse

Cette approche fonctionne quel que soit le niveau ou la disposition des téléporteurs. On peut facilement ajouter plusieurs paires de téléporteurs sans modifier le moteur de jeu.

Avantage

Le moteur reste générique et maintenable, et chaque téléporteur peut fonctionner de manière autonome, simplifiant l'extension du jeu et le polymorphisme avec `isTeleport`.

```
otherTeleport
  ^ self board teleports
    detect: [:t | t ~~ self ]
    ifNone: [ nil ]
```

COOLDOWN : ÉVITER LES BOUCLES INFINIES

Problème :

Sans précaution, un joueur pourrait être téléporté en boucle entre deux téléporteurs ($A \rightarrow B \rightarrow A \rightarrow \dots$), ce qui bloque le gameplay.

Solution :

On utilise un flag `teleportCooldown` sur le joueur pour signaler qu'une téléportation vient d'avoir lieu.

Fonctionnement :

Lorsqu'un joueur est téléporté, le flag est activé. Pendant ce temps, toute tentative de téléportation suivante est ignorée. Après la mise à jour du plateau ou du tour de jeu, le flag peut être réinitialisé.

Avantage :

Cette solution est simple, robuste et sûre : elle empêche les boucles infinies tout en conservant le flux normal du jeu. Elle rend la logique des téléporteurs prévisible et facile à maintenir.



Teleport Tiles

▼

► Multilingual-Encodings

► Myg-Compass

► Myg-Memory

► Myg-MineSweeper

► Myg-MineSweeper-Tests

► Myg-SameGame

► Myg-Sokoban

► Myg-Sokoban-Tests

► Myg-Takuzu

► Myg-Takuzu-Tests

► MygCore

► MygCore-Tests

► NECompletion

► NECompletion-Morphic

► NECompletion-Preferences

► NECompletion-Tests

► Native-Browser

Filter...

Class: MygSkTeleport

MygSkGameManager

MygSkLevelInformation

MygSkLevelPack

▼ MygSkObject

▼ MygSkGround

MygSkTeleport !

▼ MygSkMovable

▼ MygSkBox

MygSkBoxOnTarget

▼ MygSkPlayer

MygSkPlayerOnTarget

MygSkTarget

MygSkWall

MygToploPaging !

PairStrategy !

Sokoban

Filter...

► instance side .

as yet unclassified

accessing

action

testing

overrides

▲ bringIn:going:

▲ isTeleport

otherTeleport

partner

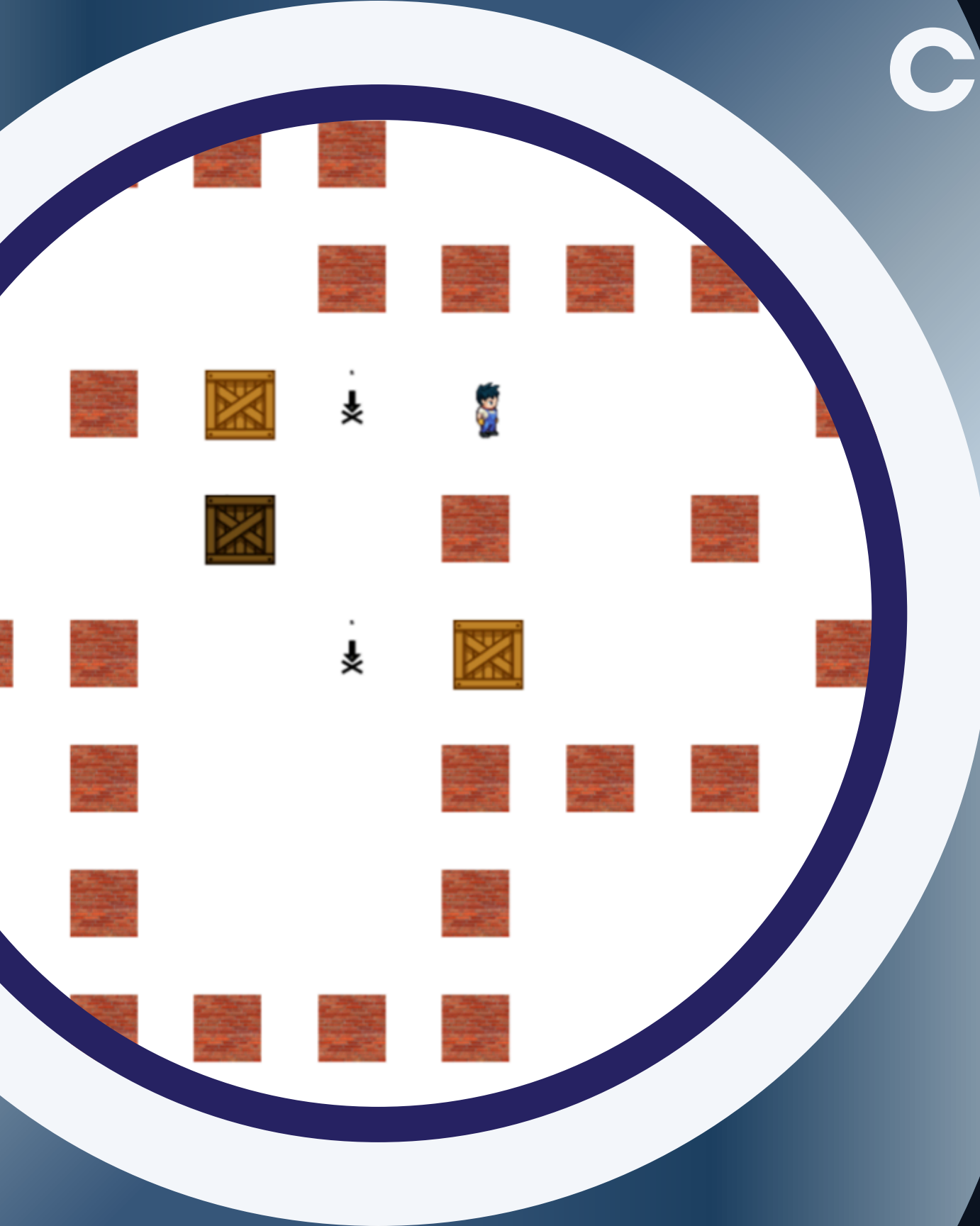
teleportPlayer:

All Packages ○ Scoped View ○ Projects | ● Flat ○ Hier. | ● Inst. side ○ Class side | ● Methods ○ Vars | Class refs.

Dependencies × MygSkTelepor × ! Comment × + Inst. side meth ×

MygSkGround << #MygSkTeleport
slots: { #partner };
tag: 'Model';
package: 'Myg-Sokoban'

Comptage des mouvements et des pushes



Concept

On suit le nombre de déplacements et de poussées pour donner un retour sur la progression du joueur.

Moves

Chaque déplacement du joueur, même sans pousser de caisse, est comptabilisé.

Pushes

Chaque fois qu'une caisse est poussée, le compteur de pushes est incrémenté.

Intérêt

Permet d'évaluer la performance du joueur et d'ajouter un challenge supplémentaire dans les niveaux.

Découplage du mouvement avec move:

Concept

La méthode move: du joueur est découplée de la logique spécifique des tuiles (y compris les téléporteurs).

Délégation

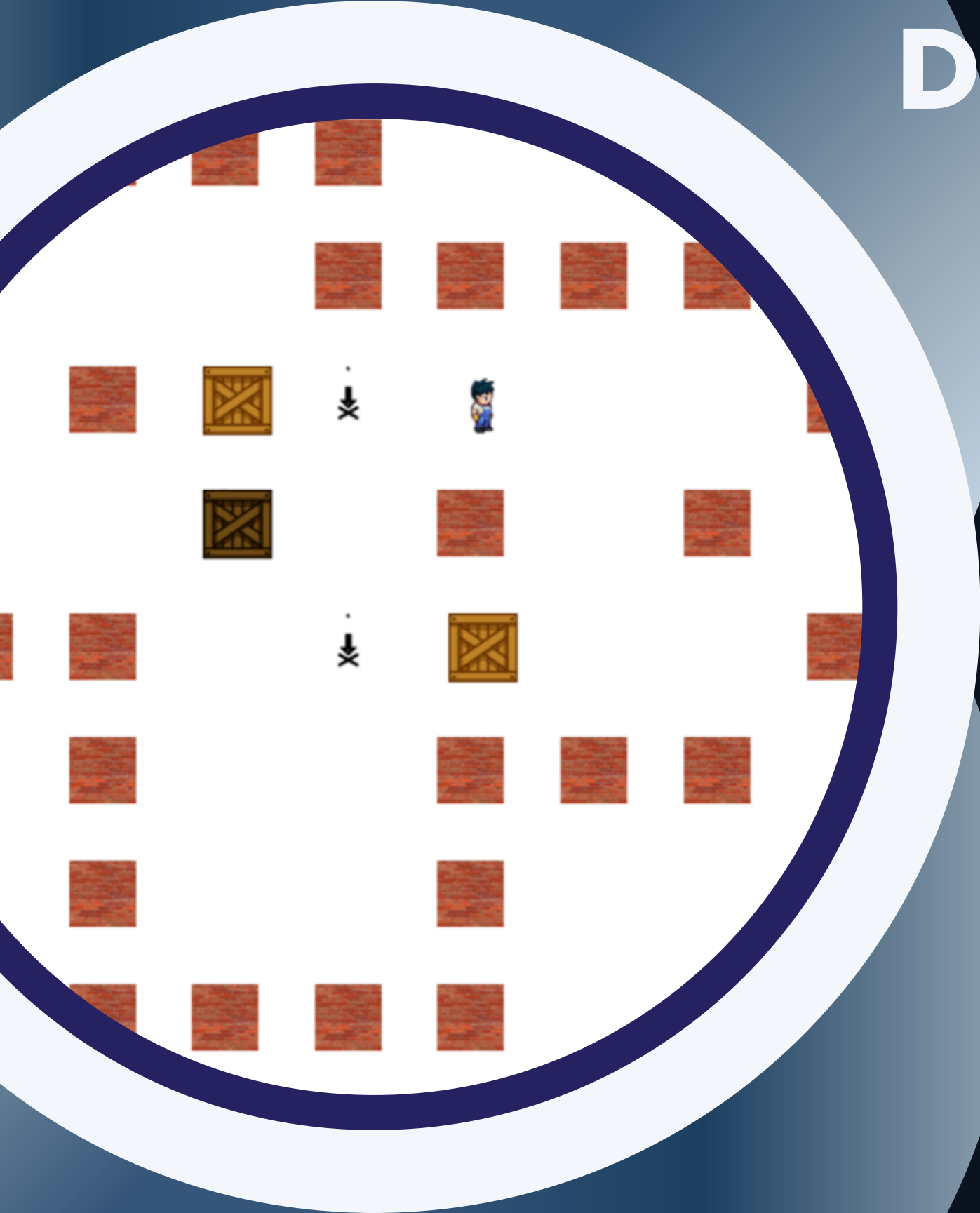
Elle demande simplement à la tuile cible de gérer l'arrivée via bringIn:going:.

Flux de contrôle

Si bringIn: retourne true, le mouvement est validé ; sinon, il échoue. Le joueur n'a pas besoin de connaître les détails de la tuile.

Avantage

Cette approche rend le moteur flexible et extensible : ajouter de nouvelles tuiles ou comportements (ex: téléport, cibles numérotées) ne nécessite pas de modifier le joueur



manageEvent: : Le déclencheur final

Concept

manageEvent: gère les événements clavier et déclenche les actions du joueur, comme déplacer, réinitialiser ou quitter le plateau.

Coordination avec move:

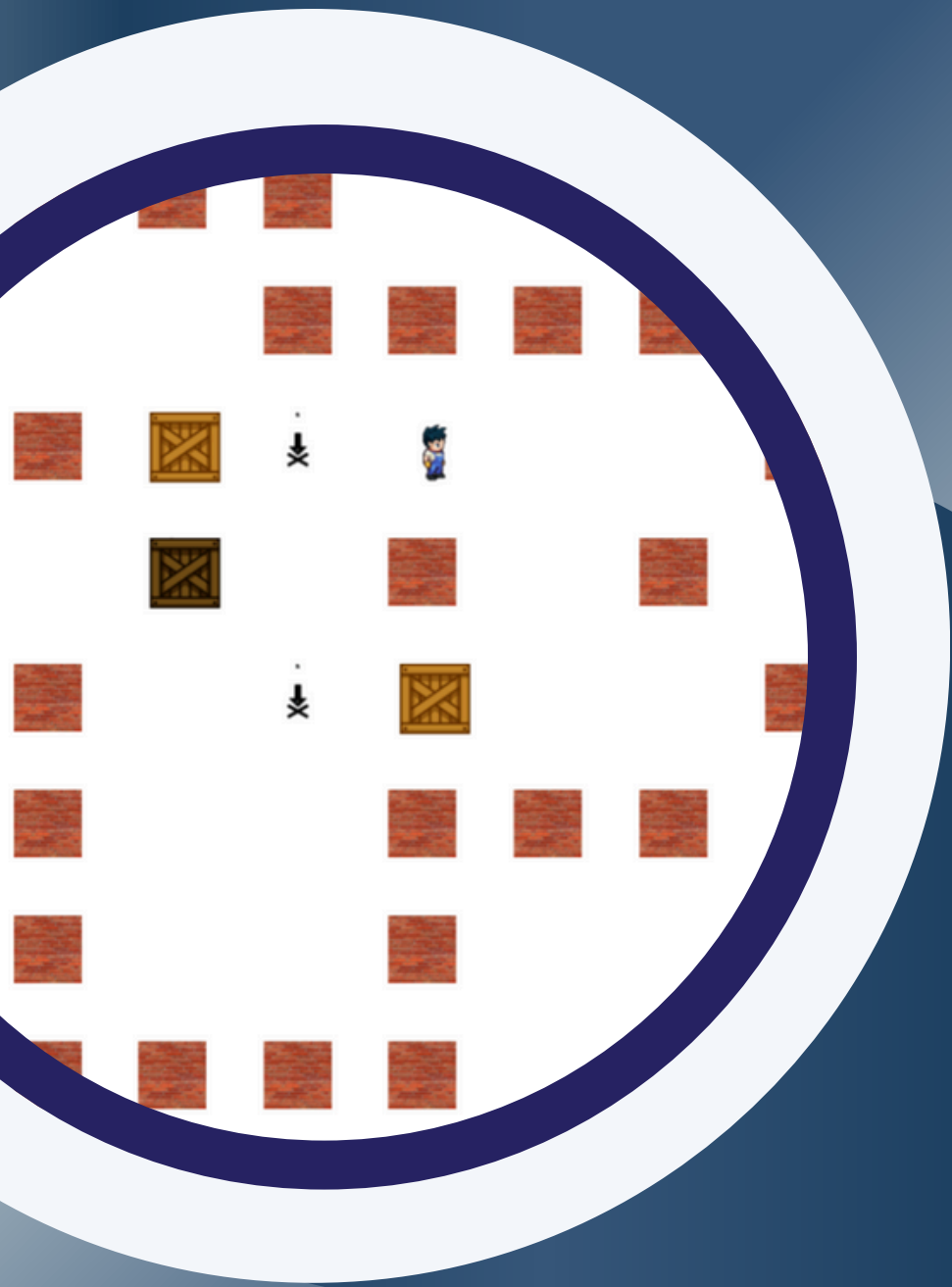
Après la capture de la direction, le joueur appelle move:, qui délègue la logique de déplacement à la tuile cible via bringIn:going:.

Mise à jour des compteurs et animations

Si le mouvement réussit, le compteur de moves est incrémenté et affiché. Les animations des caisses sont lancées si nécessaire.

Téléportation et éléments voisins

Le système vérifie si une téléportation est nécessaire après le mouvement et met à jour tous les éléments affectés pour maintenir le plateau cohérent.



manageEvent: : Le déclencheur final

```
manageEvent: anEvent

| player moved direction neighborElement elements |

"0000000 / 0000"
anEvent key name = #SPACE ifTrue: [ ^ self resetBoard ].
anEvent key name = #ESCAPE ifTrue: [ ^ self space close ].
board isFinished ifTrue: [ ^ self ].

"1. 00000000"
player := self board player.
direction := self directionDict
    at: anEvent key name
    ifAbsent: [ ^ self ]. "000000000000"

"2. 0000000000 + 000"
elements := self
    elementsToUpdateAtPosition: player position
    goingIn: direction.
neighborElement := elements second.

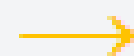
"3. 0000000 / 000"
moved := player move: direction.
```

```
moved ifTrue: [
    self board moveCount: self board moveCount + 1.
    moveLabel text: ('Moves: ', self board moveCount asString) asRopedText
].

(moved and: [ neighborElement isTeleport ]) ifTrue: [
    self teleportPlayer: player fromTeleport: neighborElement.
    ^ self
].

(moved and: [ neighborElement box class = MygSkBox ])
ifTrue: [
    | animation |
    animation := neighborElement
        boxAnimationMovingInDirection: direction.
    neighborElement addAnimation:
        (animation onFinishedDo: [
            neighborElement resetAnimation.
            self updateElements: elements ]) ]
ifFalse: [
    self updateElements: elements ].
```

✓ Verification Post-Movement



👉 Déclenchement de action

Counting Moves

The screenshot displays the Smalltalk IDE interface for the class `MygSkGameManager`. The left sidebar shows a project tree with various packages, including `Myg-Sokoban`. The central pane shows the class hierarchy, with `MygSkGameManager` selected. The right pane shows the instance variables of the class, including `currentBoardElement`, `currentBoardElement:`, `currentLevel`, `currentLevel:`, `currentLevelString`, `currentLevelString:`, `currentMoveCount`, `currentMoveCount:`, `initialize`, `initializePackDictionary`, `launchAgain`, `launchFromString:`, `launchFromString:inDictionary:at:`, `menu`, `menu:`, `notebook`, `notebook:`, and `packDictionary`.

Below the IDE, the following Smalltalk code is shown:

```
Object << #MygSkGameManager
slots: { #menu . #notebook . #currentLevelString . #currentBoardElement . #currentMoveCount . #currentLevel . #packDictionary };
tag: 'Model';
package: 'Myg-Sokoban'
```

Paired target/Box

Multilingual-Encodings

Myg-Compass

Myg-Memory

Myg-MineSweeper

Myg-MineSweeper-Tests

Myg-SameGame

Myg-Sokoban

Myg-Sokoban-Tests

Myg-Takuzu

Myg-Takuzu-Tests

MygCore

MygCore-Tests

NECompletion

NECompletion-Morphic

NECompletion-Preferences

NECompletion-Tests

Native-Browser

filter...

MygSkGameManager

MygSkLevelInformation

MygSkLevelPack

MygSkObject

MygSkGround

MygSkTeleport

MygSkMovable

MygSkBox

MygSkBoxOnTarget

MygSkPlayer

MygSkPlayerOnTarget

MygSkTarget

MygSkWall

MygToploPaging

PairStrategy

Sokoban

filter...

Class: PairStrategy

instance side

testing

isPaired

isUnpaired

pairId

pairId:

All Packages

Scoped View

Projects

Flat

Hier.

Inst. side

Class side

Methods

Vars

Class refs.

Dependencies

PairStrategy

Comment

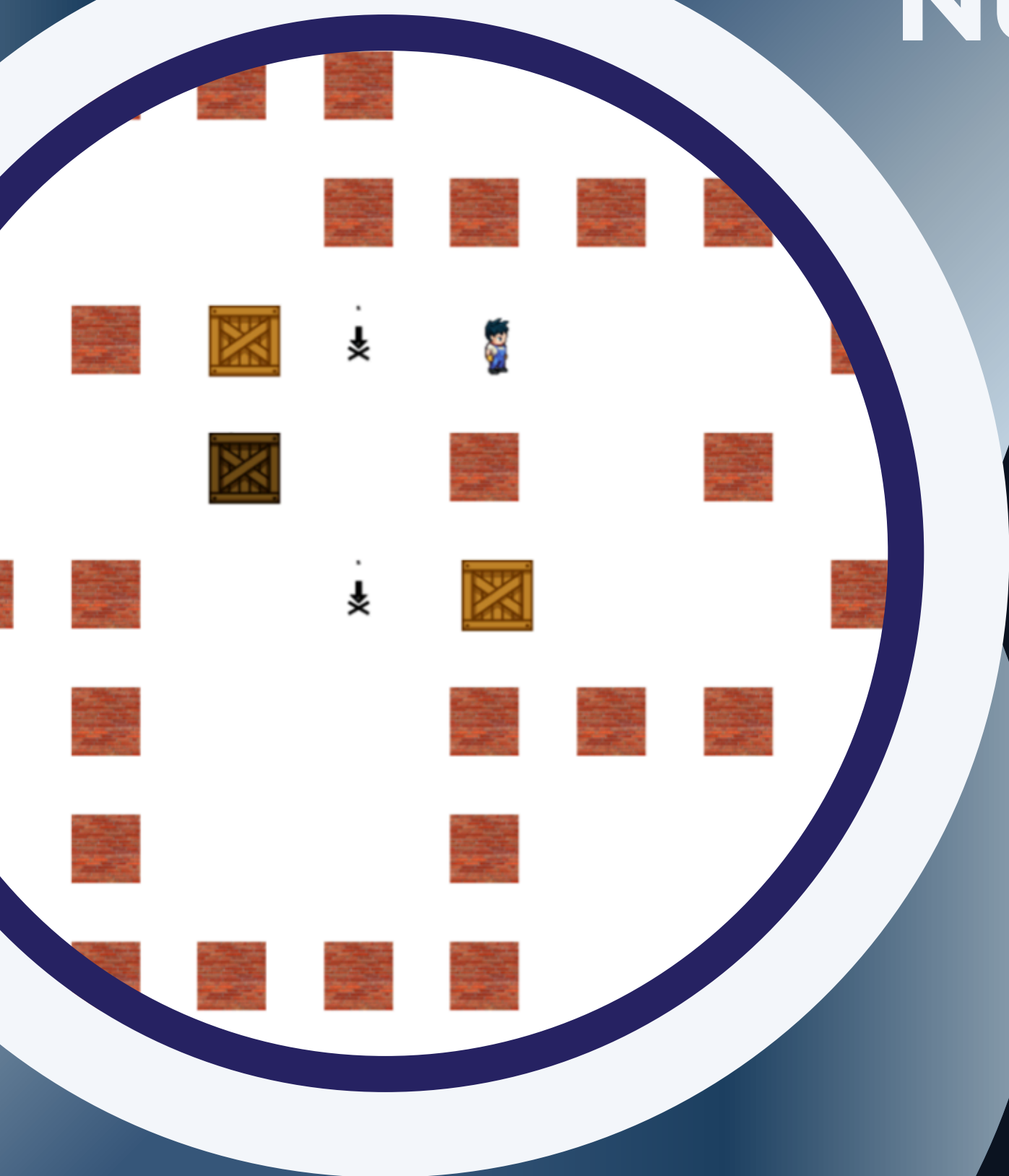
Inst. side meth

Object << #PairStrategy

slots: { #pairId };

package: 'Myg-Sokoban'

Numbered Targets : concept



Concept

Chaque cible du plateau peut être numérotée (1, 2, 3...), indiquant l'ordre dans lequel les caisses doivent y être placées.

Règle

Une caisse ne peut être validée que si elle est posée sur la cible correspondant à son ordre.

Intérêt

Cette mécanique ajoute un challenge supplémentaire, car le joueur doit planifier ses déplacements et pousser les caisses dans le bon ordre.

Avantage

Les puzzles deviennent plus stratégiques et permettent d'exploiter pleinement le moteur de jeu existant.

Target/Box : concept

Concept

Chaque boîte possède un identifiant (pairId) et ne peut être placée que sur la cible (Target) qui partage le même identifiant

Règle

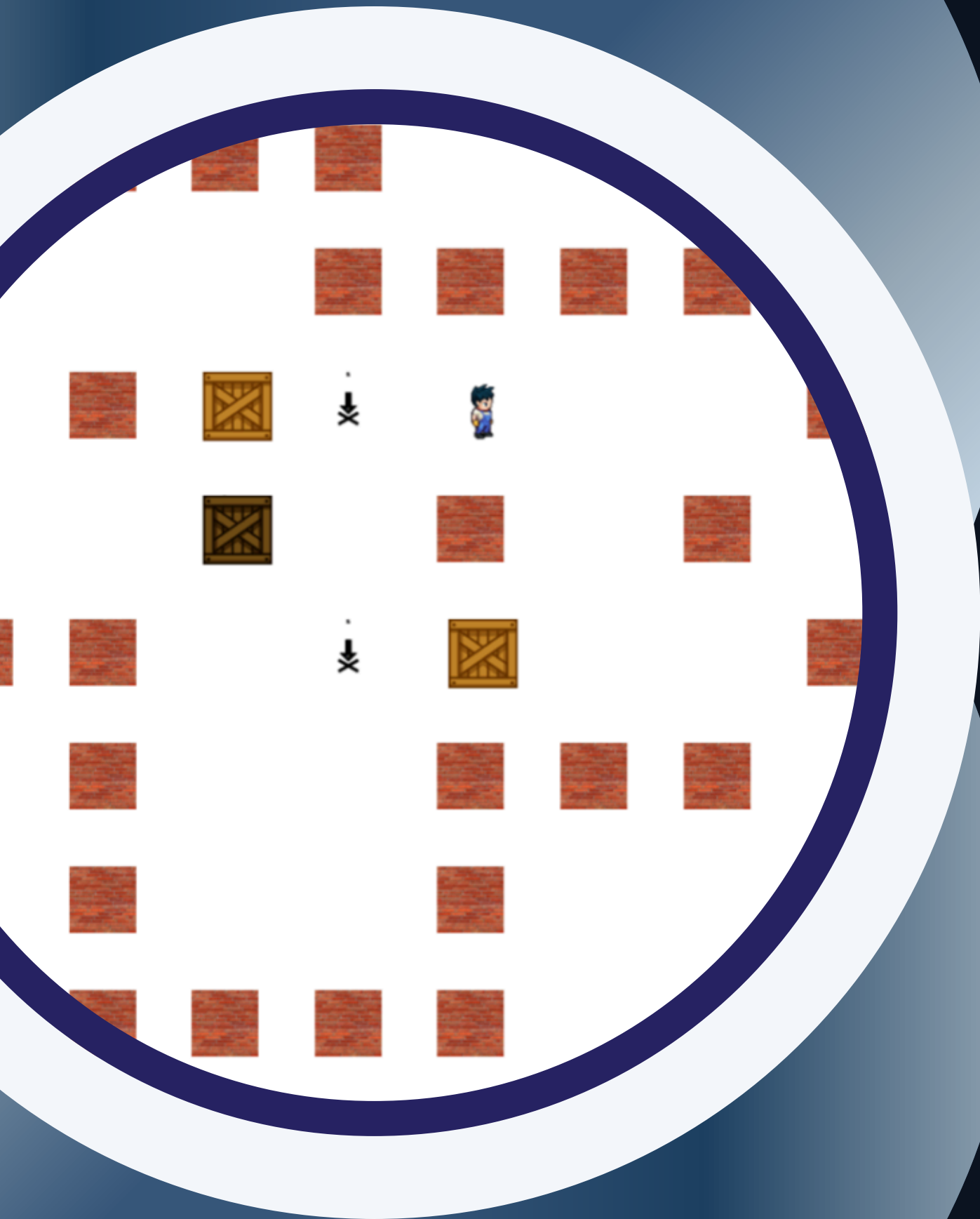
Une boîte est considérée comme correctement placée uniquement si elle est sur une cible et que leurs pairId sont égaux.

Intérêt

Cette mécanique ajoute une contrainte logique supplémentaire au jeu : le joueur ne doit pas seulement pousser les boîtes, mais aussi réfléchir à leur destination spécifique.

Avantage

Les puzzles deviennent plus stratégiques et permettent d'exploiter pleinement le moteur de jeu existant.



Numbered Targets : implémentation

Cibles numérotées

Une nouvelle classe NumberedTargetTile contient un attribut number pour indiquer l'ordre de la caisse attendue.

Vérification

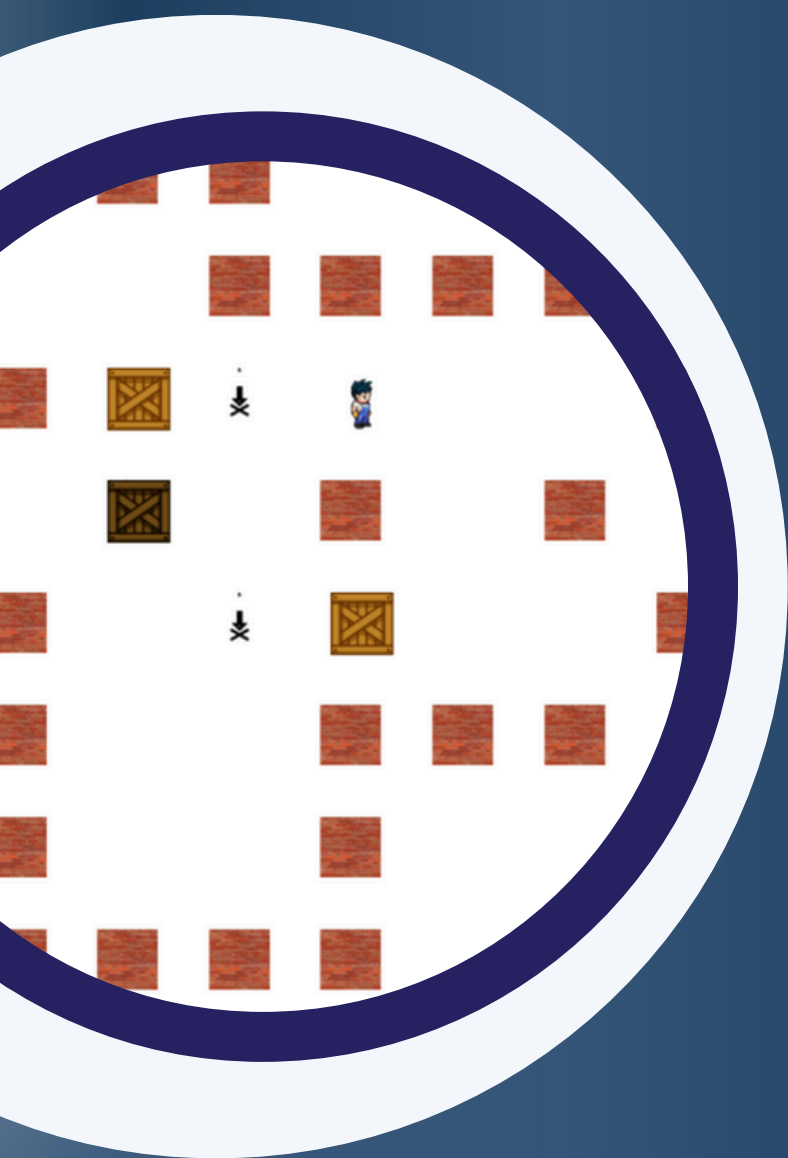
Lorsqu'une caisse est poussée sur la cible, le moteur vérifie que le numéro correspond. Sinon, la caisse n'est pas validée.

Flux

Cette vérification s'intègre dans bringIn: et respecte le découplage existant, comme pour les téléporteurs.

Extensibilité

La fonctionnalité des boîtes appariées rend le jeu plus stratégique en imposant des correspondances précises entre boîtes et cibles..



Design d'avant:

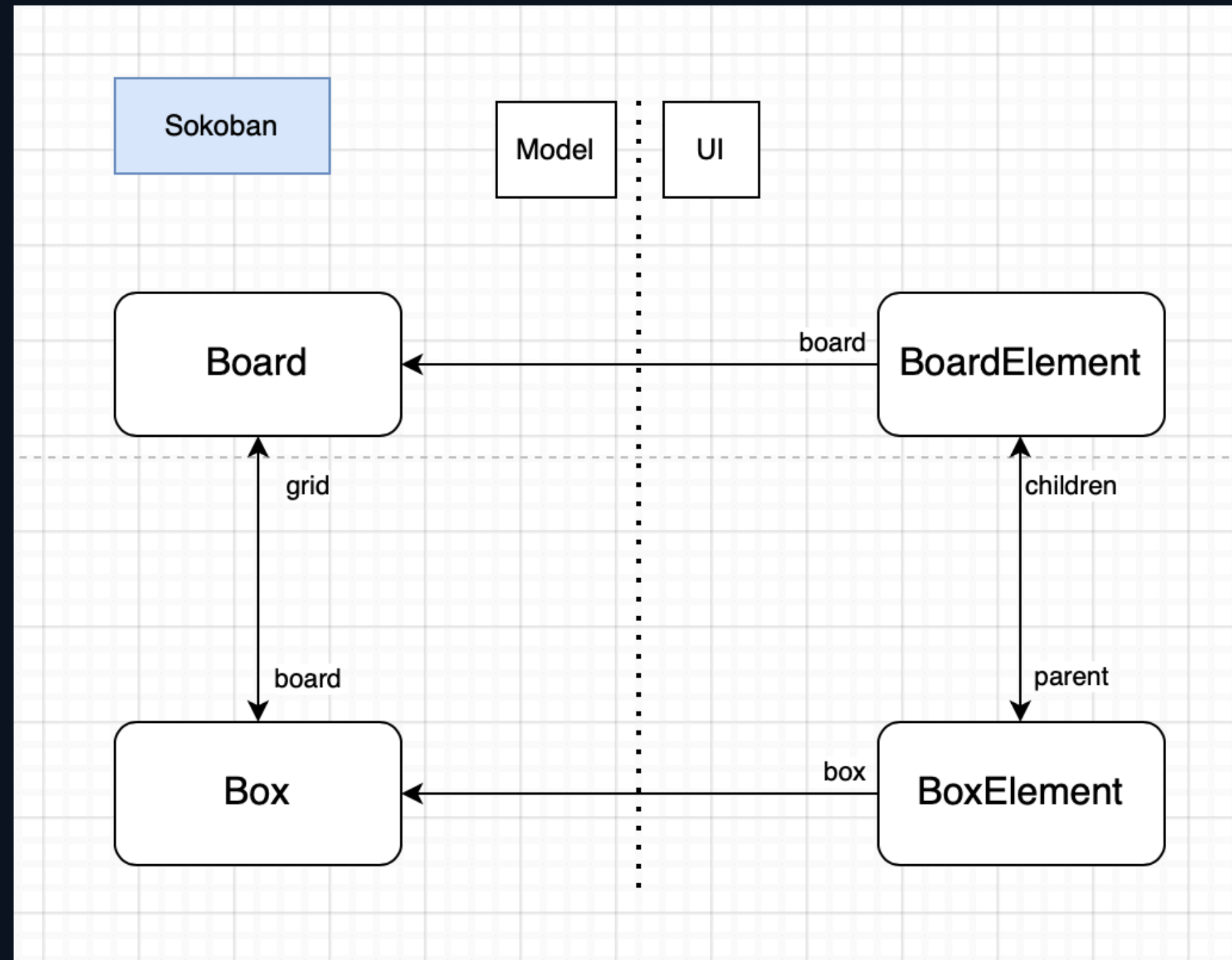
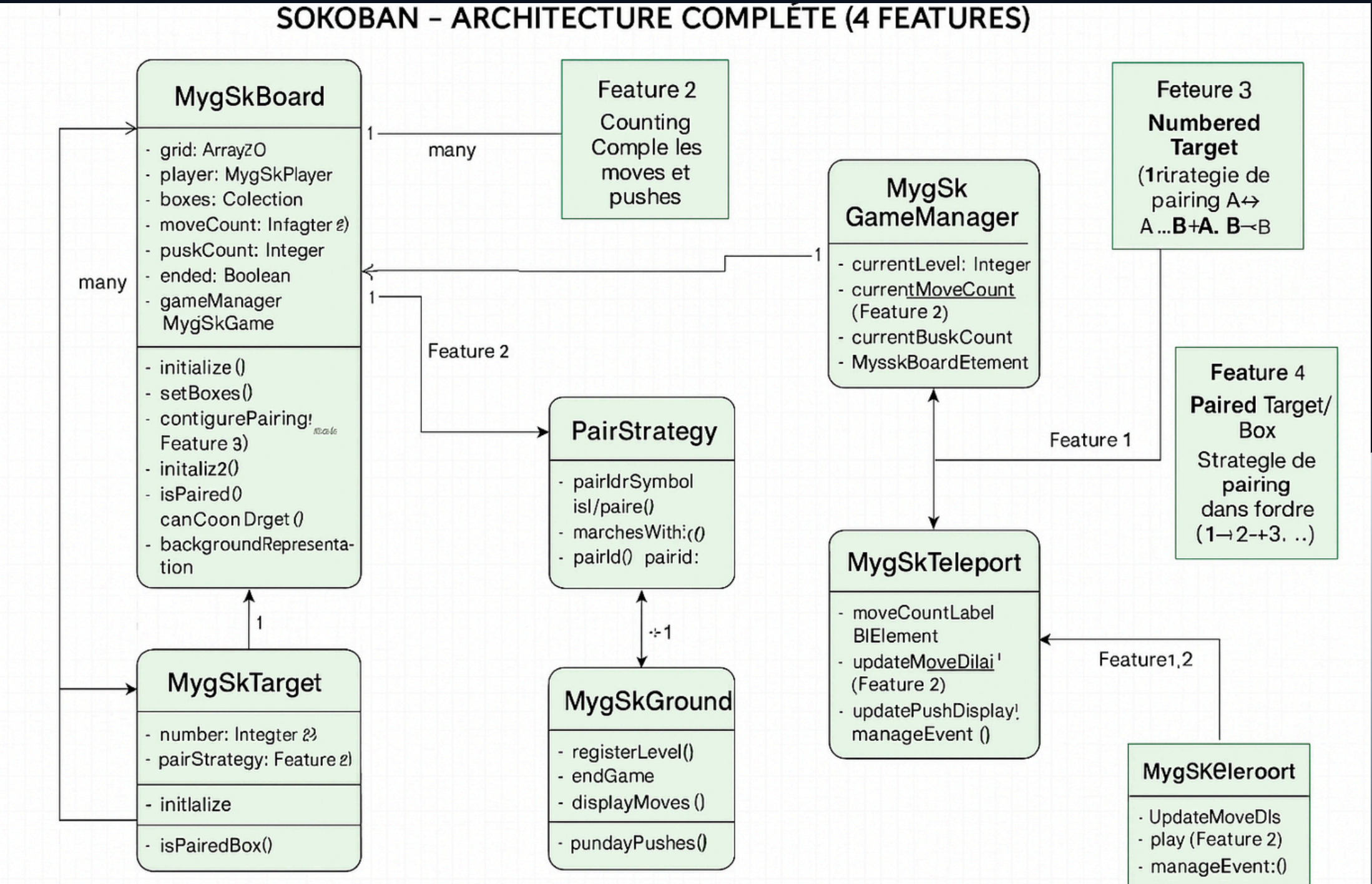


Diagramme de classe après implementation



TESTS AUTOMATISÉS

Couverture fonctionnelle

On teste tous les aspects du jeu : mouvements du joueur, poussées de caisses, collisions, téléporteurs et cibles numérotées.

Tests unitaires et d'intégration

On vérifie le comportement des tuiles et du joueur (unitaires) ainsi que la logique globale du plateau et du moteur (intégration).

Scénarios réels

On simule des niveaux complets pour s'assurer que les moves, pushes et téléportations sont correctement comptés et que le jeu se termine correctement.

Bénéfice

On garantit un moteur robuste, maintenable et extensible, facilitant l'ajout de nouvelles mécaniques sans introduire de régressions.

Myg-Sokoban

Myg-Sokoban-Tests

MygSkGameLogic

MygSkGameManagerTest

MygSkMoveCountTest

MygSkMoveInSituationTest

MygSkStructuralBoardTest

instance side

tests

tests - basic

tests - pushing box

tests - simple move

Method: MygSkMoveInSituationTest>>testPushingABoxAlreadyOnTarget

testBasicMoveInformation

testBoxBumpingInBox

testBoxBumpingInWall

testBoxMovingOnGround

testBoxMovingOnTarget

testBoxMovingOnTargetAndRestore

testPlayerBumpingInWall

testPlayerMovingOnGround

testPlayerMovingOnGroundRestoreCorrectlyGround

testPlayerMovingOnTarget

testPlayerMovingTwoTimesOnGroundRestoreCorrectlyGround

testPushingABoxAlreadyOnTarget

testPushingABoxOnGround

testPushingABoxOnTarget

testPushingABoxOnWall

SOKO

Filter...

All Packages

Scoped View

Projects

Flat

Hier.

Inst. side

Class side

Methods

Vars

Class refs.

Implementors

Senders

Dependencies

MygSkTelepori

MygSkMoveInS

Comment

setUp

*textualMapCh

*manageEvent

testPushingAB

Inst. side meth

testPushingABoxAlreadyOnTarget

| tmp1 tmp2 |

tmp1 := MygSkBoard createWithBoard:

((MygLevelImporter

fromString: '#@* #'

fromPackage: MygSkBoard package) collect: [:arg1 |

arg1 configured]).

tmp2 := tmp1 player position.

self

assert: tmp1 asciiString equals: '#@* #';

assert: tmp1 player moveEast;

assert: tmp1 player position x equals: 3;

assert: (tmp1 atPosition: 3 @ 1) isPlayer;

assert: tmp1 asciiString equals: '# +\$#';

deny: (tmp1 atPosition: 2 @ 1) isPlayer

Method: MygSkStructuralBoardTest>>testAtPositionLo

Myg-Sokoban

Myg-Sokoban-Tests

MygSkGameLogic

MygSkGameManagerTest

MygSkMoveCountTest

MygSkMoveInSituationTest

MygSkStructuralBoardTest

instance side

data

tests

tests - building

map1

testAtPositionLowLevel

testIsBoxOnTargetPlaygroundCorrectlySet

testIsBoxPlaygroundCorrectlySet

testIsPalyerOnTargetPlaygroundCorrectlySet

testIsPlayerPlaygroundCorrectlySet

testPlayerIsAvailable

testPrintOn

testPrintOnOneLine

SOKO

Filter...

All Packages

Scoped View

Projects

Flat

Hier.

Inst. side

Class side

Methods

Vars

Class refs.

Implementors

Senders

Dependencies

MygSkTeleport

MygSkStructur

Comment

setUp

*textualMapCh

*manageEvent

testAtPositionl

testAtPositionLowLevel

| board |

board := MygSkBoard createWithBoard:

((MygLevelImporter

fromString: self map1

fromPackage: MygSkBoard package) collect: [:each | each configured]).

self assert:

((board atPosition: CpDirection north * board player position)

isKindOf: MygSkGround).

self assert:

((board atPosition: CpDirection south * board player position)

isKindOf: MygSkWall).

self assert:

((board atPosition: CpDirection east * board player position)

isKindOf: MygSkWall).

self assert:

((board atPosition: CpDirection west * board player position)

isKindOf: MygSkWall)

THANK YOU

